Conditionals in Python

if | elif | else | comparisons | logic | ternary | match

if / elif / else

Only the **first** true branch runs. The indented block under each test is its body; `else` is the catch-all.

```
score = 72
if score >= 90:
    grade = "A"
elif score >= 70:
    grade = "B"
else:
    grade = "C"
print(grade)                    # → B
```

Truthy and falsy values

Any object can be a condition. **Falsy:** `0`, `0.0`, `""`, `[]`, `{}`, `()`, `set()`, `None`, `False`. Everything else is truthy.

```
name = ""
if not name:                    # empty string is falsy
    name = "Guest"
items = [3, 4]
if items:                       # same as len(items) > 0
    print(len(items))
```

OUTPUT

```
2
```

Nested vs flat conditions

Prefer one combined test over a pyramid of nested `if`s.

```
temp = 25
if temp > 0:                    # nested
    if temp < 100:
        print("liquid")
if 0 < temp < 100:              # flat, same result
    print("liquid")
```

OUTPUT

```
liquid
liquid
```

match / case (Python 3.10+)

Structural pattern matching. `|` joins alternatives, `_` is the default, and `case _ if cond:` adds a guard.

```
cmd = "halt"
match cmd:
    case "go":
        print("Moving")
    case "stop" | "halt":
        print("Stopping")
    case _:
        print("Unknown")
```

OUTPUT

```
Stopping
```

Comparison operators

Operator	Meaning
`== !=`	equal / not equal
`< <=`	less than / at most
`> >=`	greater than / at least
`is is not`	same object (use for `None`)
`in not in`	membership: list, str, dict, set
`1 < x < 10`	chained: no `and` needed

Logical operators: and, or, not

`and` needs both, `or` needs either, `not` flips the result. They **short-circuit**: evaluation stops as soon as the answer is known.

```
age, member = 25, True
if age >= 18 and member:
    print("Access granted")
if not member or age < 18:
    print("Denied")
```

OUTPUT

```
Access granted
```

Ternary (one-line if/else)

`value_if_true if condition else value_if_false`: an expression, so it can be assigned or passed around.

```
age = 20
status = "adult" if age >= 18 else "minor"
print(status)
fee = 0 if age < 5 else 10
```

OUTPUT

```
adult
```

Membership and None checks

```
role = "admin"
if role in ("admin", "owner"):    # not: == or ==
    print("full access")
data = None
if data is None:                  # never == None
    print("no data")
```

OUTPUT

```
full access
no data
```

Classic mistakes

Wrong	Right
`if x = 5:` (SyntaxError)	`if x == 5:`
`if x == 1 or 2:` (always true)	`if x in (1, 2):`
`if flag == True:`	`if flag:`
`if x == None:`	`if x is None:`
two separate `if`s that overlap	`if` … `elif` … `else`

REMEMBER

- Every `if`, `elif` and `else` line ends with a colon, and its body is indented (4 spaces).
- Order `elif` branches from most specific to most general: the first match wins.
- `and` / `or` return one of their operands, not always `True`/`False`: `0 or 5` gives `5`.
- `and` binds tighter than `or`. Add parentheses to make mixed logic obvious.
- Use `match` for many fixed alternatives; keep `if/elif` for ranges and complex tests.
- Chained comparisons (`0 < x < 10`) read like maths and are evaluated once.